What is a DAG (Directed Acyclic Graph) in Apache Spark?

A **DAG (Directed Acyclic Graph)** is Spark's **execution plan** that represents all the transformations required to process data before any computation is executed.

Spark builds a DAG automatically from your transformations, optimizes it, and then executes it efficiently when an **action** is called.

What Does DAG Stand For?

- **Directed** → The operations have a specific execution order (parent → child).
- **Acyclic** → There are **no loops or cycles**. An operation cannot depend on itself.
- **Graph** → A network of operations (nodes) connected by dependencies (edges).

Why Does Spark Use a DAG?

Spark uses a DAG to:

- Optimize query execution.
- Minimize unnecessary computations.
- Reduce data shuffling.
- Identify operations that can run in parallel.
- Improve overall performance.

Instead of executing each transformation immediately, Spark first builds the entire execution plan and then optimizes it.

How Does a DAG Work?

Suppose you write the following code:

```
df = spark.read.csv("employees.csv", header=True)

result = (
    df.filter(df.salary > 5000)
```

```
      .select("department", "salary")
      .groupBy("department")
      .avg("salary")
)

result.show()
```

Spark does **not** execute the transformations immediately.

Instead, it builds the following DAG:

```
Read CSV
    |
    ▼
Filter
    |
    ▼
Select
    |
    ▼
GroupBy
    |
    ▼
Average
    |
    ▼
Show (Action)
```

Only when `show()` is called does Spark execute the DAG.

DAG and Lazy Evaluation

Spark transformations are **lazy**.

This means:

1. Every transformation is recorded.
2. Spark builds the DAG.
3. Spark optimizes the DAG.
4. An action triggers execution.

Without lazy evaluation, Spark would execute each transformation separately, resulting in unnecessary work.

DAG to Stages

The DAG is **not executed directly**.

Spark first divides the DAG into **stages**.

A new stage is created whenever Spark encounters a **shuffle operation**, such as:

- `groupBy()`
- `join()`
- `distinct()`
- `orderBy()`
- `repartition()`

Example

```
df.filter(df.salary > 5000) \
  .groupBy("department") \
  .count() \
  .show()
```

Execution flow:

```
DAG

Read
 |
 ▼
Filter
 |
 ▼
GroupBy  ← Shuffle
 |
 ▼
Count
 |
 ▼
Show
```

Stages:

```
Stage 1
--------
Read
Filter

      Shuffle

Stage 2
--------
GroupBy
```

```
Count
Show
```

DAG → Stages → Tasks

After creating stages, Spark divides each stage into **tasks**.

Each task processes **one data partition**.

Example:

```
Dataset
   |
200 Partitions
   |
   ▼
200 Tasks
```

Execution flow:

```
User Code
     |
     ▼
Transformations
     |
     ▼
DAG
     |
     ▼
Stages
     |
     ▼
Tasks
     |
     ▼
Executors
```

Who Creates the DAG?

The **Driver Program** creates the DAG.

Responsibilities of the Driver include:

- Reading your Spark code.
- Building the logical execution plan.
- Optimizing the plan.
- Creating the DAG.

- Splitting the DAG into stages.
- Splitting stages into tasks.
- Sending tasks to executors.

Advantages of a DAG

- Improves performance by optimizing execution.
- Reduces unnecessary computations.
- Minimizes data shuffling.
- Enables parallel execution.
- Supports fault tolerance through lineage.
- Helps Spark recover lost partitions without recomputing the entire dataset.

DAG vs Stage vs Task

Component	Description
DAG	The complete execution plan consisting of all transformations.
Stage	A group of transformations that can execute without a shuffle.
Task	The smallest unit of work that processes one data partition.

Relationship:

```
DAG
 │
 ├── Stage 1
 │       ├── Task 1
 │       ├── Task 2
 │       └── Task 3
 │
 └── Stage 2
         ├── Task 1
         ├── Task 2
         └── Task 3
```

Real-World Example

Imagine you need to analyze **500 GB** of sales data.

You perform the following operations:

1. Read the data.
2. Filter records for the current year.
3. Join with a customer table.
4. Group by country.
5. Calculate total sales.

Spark:

- Builds a DAG containing all these transformations.
- Optimizes the execution plan.
- Splits it into stages (before and after the join/shuffle).
- Creates tasks based on data partitions.
- Executes the tasks in parallel across executors.

This optimization is one of the main reasons Spark is much faster than systems that execute each step independently.

Interview Questions

Q1. What is a DAG in Spark?

A **DAG (Directed Acyclic Graph)** is Spark's execution plan that represents all transformations and their dependencies. Spark builds and optimizes the DAG before executing it.

Q2. Who creates the DAG?

The **Driver Program** creates the DAG after analyzing the transformations in your Spark application.

Q3. When is the DAG executed?

The DAG is executed **only when an action** (such as `show()`, `count()`, or `collect()`) is called.

Q4. Why is the DAG called "acyclic"?

Because it has **no cycles**. An operation cannot depend on itself, so the execution flow always moves forward.

Q5. What is the relationship between DAG, Stage, and Task?

- A **DAG** is the complete execution plan.
- A **Stage** is a section of the DAG separated by shuffle boundaries.
- A **Task** is the smallest unit of execution and processes one data partition.

Interview Summary

- A **DAG (Directed Acyclic Graph)** is Spark's optimized execution plan for processing data.
- The **Driver** builds the DAG from the application's transformations during lazy evaluation.
- When an **action** is called, Spark optimizes the DAG, splits it into **stages** at shuffle boundaries, and then divides each stage into **tasks**, with one task per data partition.
- DAG-based execution allows Spark to optimize queries, reduce unnecessary computation, minimize shuffles, execute tasks in parallel, and recover from failures efficiently through lineage.